

Synthetic Market Phase 21

Source-driven restoration across two axes on the compiled patch path: Leader (L4, top-4 components) and Dispersion (L35, top-4 components). This report replaces the earlier mixed-batch draft and uses only the validated `batch_size = 16` reruns: 48 matched pairs per axis, separately generated source behaviors, and a `swap_components` intervention that inserts source-side coefficients into the base prompt.

DATE	ROWS	PATCH	ENGINE
3 April 2026	48 per axis	swap_components, source-driven donor means	compiled non-eager custom-op

MAIN READ

The validated bs16 rerun preserves the main scientific split from the old draft while fixing the decoding pathology. Leader still shows the stronger partial restoration signal on action choice and spend. Dispersion is no longer obviously broken at the decode level, but it remains weak on action restoration and poor on response length. Phase 22 should therefore stay leader-first rather than promoting dispersion into the main path-validation target.

LEADER RESTORE	TOOL-TOKEN	DISPERSION TOOL-TO- KEN RESTORE	LEADER RESTORATION	LENGTH	DISPERSION RESTORATION	LENGTH
0.25		0.136	-1.396		-3.13	
4 of 16 restorable rows		3 of 22 restorable rows	negative means the patched response moved away from the source length		much more negative than Leader	

What Was Tested

For each matched pair, the run produced three behaviors:

- the source prompt on its own
- the base prompt on its own
- the base prompt with a `swap_components` patch

The question is simple:

- did the patch make the base prompt behave more like its matched source prompt?

This report uses denoise pairs. In plain English, that means:

- the base row is the lower-valued member of the pair on the chosen metric
- the source row is the higher-valued member of the pair

Here the chosen metric depends on the axis:

- Leader uses `vol_1h_max`
- Dispersion uses `pct_1h_mad`

`swap_components` also has a plain-English meaning here:

- it does *not* copy every hidden-state token from source to base
- it captures the source row, averages over the market span, and extracts the selected coefficients
- it then writes those source-side coefficients into the base row over the base market span

One operational note matters for interpretation:

- both axes were rerun at `batch_size = 16`
- both used the same compiled non-eager custom-op patch path
- this replaces the earlier draft whose Dispersion result was contaminated by the batch-32 tool-surface failure

Decode Validity

Before interpreting the restoration metrics, the reruns need to clear the decode-validity check that the old draft failed.

Check	Leader	Dispersion
Base rows with valid tool calls	48 / 48	48 / 48
Source rows with valid tool calls	48 / 48	48 / 48
Finish reason stop	48 / 48 base and source	48 / 48 base and source
15000-token cap hits	0	0
!!!! corruption rows	0	0
Missing <code></think></code> rows	0	0

So the key methodological upgrade over the old report is not only the science. It is that both axes are now being evaluated on a clean, validated generation regime.

Axis Comparison

Measure	Leader	Dispersion	How to read it
Source tool-name match	0.979 → 1	1 → 1	Leader improves from near-satura-

Measure	Leader	Dispersion	How to read it
			tion; Dispersion stays saturated
Source tool-token match	0.667 → 0.708	0.542 → 0.521	Leader improves; Dispersion is slightly negative even after the clean rerun
Tool-token restoration rate	0.25	0.136	Higher is better on rows that needed fixing
Tool-token backfire rate	0.063	0.154	Lower is better on rows that were already correct
Spend improvement rate	0.667	0.6	Both axes move spend toward the source more often than not; Leader still has the clearer action result
Generated-token improvement rate	0.438	0.396	Both are weak; Leader is still less bad
Mean normalized length restoration	-1.396	-3.13	Negative means the patch moved away from the source

Restorable Rows And Backfires

Metric	Leader	Dispersion
Tool name	restorable 1 restored 1 backfires 0 / 47	restorable 0 restored 0 backfires 0 / 48
Tool token	restorable 16 restored 4 backfires 2 / 32	restorable 22 restored 3 backfires 4 / 26
Spend pct	restorable 3 improved 2 backfires 2 / 47	restorable 5 improved 3 backfires 1 / 48
Generated tokens	restorable 48 improved 21 backfires 27 / 48	restorable 48 improved 19 backfires 29 / 48

This table is the clearest practical summary:

- Leader helps on tool-token choice more often than it hurts
- Dispersion hurts tool-token choice almost as often as it helps

- Dispersion spend is now real on a small five-row restorable subset, but the action-surface result is still weak
- neither axis gives a clean length-restoration story

Mechanical Validity

Measure	Leader	Dispersion
Rows with patch stats	48	48
Patch applied rate	1	1
Patch skipped rate	0	0

Mechanically, both runs are clean. Every row has patch diagnostics and nothing was skipped.

Methodology Scorecard

Best-practice item	Status
Matched prompt pairs and separately generated source behaviors	meets
Source-driven coefficient swap rather than generic ablation	meets
Compiled non-eager custom-op execution path for the patch itself	meets
Per-row patch diagnostics with zero skips on both axes	meets
Exact token-by-token source activation transplant	missing
Clean restoration of action choice on every tested axis	partial
Clean restoration of response length / verbosity	missing
Path-level validation	missing

The main methodological upgrade over Phase 20 is that Phase 21 uses *generated source behaviors* and a *source-driven swap*. That makes this a real restoration test, not only a disruption test.

The main methodological upgrade over the earlier Phase 21 draft is that this version also clears a *decode-validity* check: both axes were rerun in a batch regime with no degenerate decodes, so the axis comparison is now operationally apples-to-apples.

Interpretation

Leader and Dispersion still do not behave the same way.

Leader is the better causal candidate:

- it improves source match on the action surface we care about most
- tool-token restoration is materially above backfire
- spend moves toward the source more often than not

Dispersion is cleaner than it looked in the old draft, but it is still weaker:

- tool-name match is saturated, so there is no tool-name restoration story to tell

- tool-token match is slightly worse overall after the patch
- tool-token restoration and backfire are roughly the same size
- spend does move toward the source on a five-row restorable subset, but that is not enough to promote Dispersion into the main causal handle

The combined takeaway is not “restoration failed.” It is more specific:

- the market representation is not uniform
- Leader looks more like a usable causal handle on action choice
- Dispersion may still matter for a narrower calibration-like quantity, but not for clean action restoration
- neither axis yet restores whole-response length cleanly

Next Step

The next experiment should push the restoration story where it is currently strongest:

- keep `batch_size = 16` for this Qwen3 thinking + forced-tool setup unless batch invariance proves 32 stable
- run Phase 22 path validation on *Leader first*
- keep Dispersion as a contrast case or calibration follow-up, not as the primary Phase 22 path-validation target
- if possible, move from span-mean coefficient swapping toward a more exact source-activation transplant

If Leader keeps restoring tool-token choice under those stronger tests, the causal claim becomes materially harder to dismiss.